# S2M

*Solutions Monétiques*

Référentiel Technique

# Fondation des projets

*Stack · Structure · Règles*

| Élément | Valeur |
|---|---|
| Document | Référentiel Technique S2M |
| Version | 2.0 |
| Date | Avril 2026 |
| Audience | Claude Code (exécution) · Claude chat (guidance) · Architectes · Développeurs |
| Portée | Tous les projets S2M nouveaux ou refondus |
| Format | Règles impératives · Templates copiables · Catalogue de packages |

# 0. Usage du document

### Audience et objet

- Document utilisé comme référence par Claude Code (génération de code) et Claude chat (orientation et discussion).
- Format prescriptif : règles directes sans justifications longues.
- Contexte spécifique d'un projet (état d'avancement, modules existants, décisions ad-hoc) : voir PROJECT_CONTEXT.md du projet, jamais ici.
- Ce référentiel évolue trimestriellement ou sur évolution majeure de stack.

### Pour Claude Code

- Lire ce doc et le CLAUDE.md du projet en début de session.
- Aucune règle MUST/MUST NOT ne peut être contournée sans validation explicite humain.
- En cas de conflit entre ce doc et le code existant, signaler avant d'agir.
- Tout pattern non listé ici nécessite validation humain avant introduction.

# 1. Stack technique

**Synthèse :** Node.js 24 LTS + NestJS 11 + TypeScript 6 (backend), Next.js 16 + React 19 + Tailwind 4 (frontend), PostgreSQL 18 + Redis 7 + NATS JetStream 2.10 (datastores). Versions figées au 28 avril 2026.

## 1.1 Runtime et langage

| Outil | Version | Rôle |
|---|---|---|
| Node.js | 24 LTS "Krypton" (24.15.0) | Runtime backend. LTS jusqu'en avril 2028. npm v11. Worker Threads stables. Support TS natif. |
| TypeScript | 6.0.3 | Backend + frontend. Mode strict, target ES2025, ESM. Sortie 23 mars 2026. |

## 1.2 Backend

| Outil | Version | Rôle |
|---|---|---|
| NestJS | 11.1.19 | Framework backend. Modules, DI, OpenAPI. SWC compiler par défaut. Vitest natif. |
| Drizzle ORM | 0.45.2 | ORM TypeScript-first. Accès DB par défaut sur tout le code applicatif. Support natif PG 18. |
| pg (node-postgres) | 8.13+ | Driver bas niveau. Réservé migrations ad-hoc et cas validés en revue. Pas d'usage applicatif courant. |
| ioredis | 5.4+ | Client Redis. Locks distribués, idempotence, vélocité, sessions. |
| nats.js | 2.28+ | Client NATS JetStream. Publication/souscription événements durables. |
| decimal.js | 10.4+ | Calcul décimal exact. number natif INTERDIT pour valeurs monétaires. |
| zod | 4.3.6 | Validation runtime + types partagés via workspace shared. 14× plus rapide que v3. |
| pino | 10.3.1 | Logger JSON structuré. Corrélation traces OpenTelemetry par trace_id. |
| OpenTelemetry SDK Node | 1.x latest | Observabilité backend. Auto-instrumentation HTTP, pg, ioredis, NestJS. |
| Vitest | 2.1+ | Tests unitaires et intégration. Intégré à NestJS 11. |

| Outil | Version | Rôle |
|---|---|---|
| Supertest | 7.0+ | Tests HTTP des controllers NestJS. |

## 1.3 Frontend

| Outil | Version | Rôle |
|---|---|---|
| Next.js | 16.2.4 LTS | Framework React. App Router, Server Components, Turbopack par défaut. |
| React | 19.2 | Lib UI. View Transitions, useEffectEvent, Activity. React Compiler stable. |
| TypeScript | 6.0.3 | Identique au backend. Schémas zod partagés via shared. |
| Tailwind CSS | 4.2.4 LTS | Utility-first. Configuration via @theme. Build 3,5× plus rapide que v3. |
| shadcn/ui | latest | Composants headless sur Radix UI. Code dans le projet, modifiable avec validation. |
| TanStack Query | 5.51+ | State serveur côté client. Cache, mutations, invalidations. |
| React Hook Form | 7.52+ | Formulaires. Intégration zod via @hookform/resolvers. |
| Playwright | 1.48+ | Tests E2E multi-navigateurs. |
| Vitest | 2.1+ | Tests unitaires React via @testing-library/react. |
| OpenTelemetry Web | 0.x latest | Observabilité client. Capture erreurs JS, propagation trace_id. |
| next-intl | latest | Internationalisation. EN + FR par défaut, autres locales ajoutables. |
| SELECT-PX Design System | @s2m/select-px-design | Design system parent S2M. Tokens Tailwind 4 (@theme), composants Brand, shadcn customisé. Voir §4.4. |

## 1.4 Datastores

| Outil | Version | Rôle |
|---|---|---|
| PostgreSQL | 18.3 | Source de vérité unique. AIO, uuidv7, VIRTUAL columns, RETURNING OLD/NEW. Support 2030. |
| Redis | 7.4 | Cache, locks, idempotence, vélocité, sessions. Aucune donnée critique. |

| Outil | Version | Rôle |
|---|---|---|
| NATS JetStream | 2.10+ | Bus d’événements persistants. Découple producteurs/consommateurs. |
| Flyway | 10.20+ | Migrations DB versionnées via fichiers SQL numérotés. |

# 2. Interactions entre layers

## 2.1 Schéma

```
Frontend (Next.js)
   |  REST + JWT (Authorization header)
   |  payloads validés via zod (workspace shared)
   |  trace_id propagé en header traceparent
   v
Backend (NestJS, hexagonal)
   +-------------------------------------------------------+
   |  domain  <-  application  <-  ports                   |
   |                               ^                       |
   |                               |                       |
   |                           adapters                    |
   |                       +------+------+                 |
   |                       v      v      v                 |
   |                      Drizzle ioredis nats.js          |
   +-----------+----------------+----------------+-------+
               |                |                |
       transactions       cache, locks         events
               v                v                v
         PostgreSQL          Redis        NATS JetStream
         (source vérité)     (éphémère)   (persistant)
```

## 2.2 Frontend → Backend

- Communication exclusivement via REST sur HTTPS.
- JWT envoyé en header Authorization: Bearer <token>.
- Validation token côté backend via NestJS Guards (@s2m/core-auth).
- Payloads validés des deux côtés avec le même schéma zod (shared package).
- trace_id OpenTelemetry propagé en header traceparent (généré côté client si absent).
- Backend adopte le trace_id comme racine pour corréler logs + SQL + NATS.

## 2.3 Backend → Datastores

- **PostgreSQL** : opérations d'état durable. Transactions ACID pour opérations critiques.
- **Redis** : éphémère uniquement (verrouillage, idempotence, cache, vélocité).
- **NATS** : publication d'événements. Consommateurs indépendants.
- Le backend ne lit jamais Redis ou NATS comme source de vérité. Seul PostgreSQL fait foi.

## 2.4 Datastores entre eux

- Aucune interaction directe entre datastores.
- Le backend orchestre toutes les coordinations.

- Pattern type : lock Redis → transaction PostgreSQL → release Redis → publication NATS.

## 2.5 Cohérence transactionnelle

- ACID garantie uniquement à l'intérieur d'une transaction PostgreSQL.
- Au-delà : eventual consistency.
- **Pattern outbox obligatoire pour événements critiques** : événement écrit en table PostgreSQL au sein de la transaction, relais asynchrone publie sur NATS.

# 3. Structure du dépôt

## 3.1 Approche : monorepo applicatif

- Un seul dépôt Git par projet : backend + frontend + shared + migrations + deploy.
- Workspaces pnpm. Refactors atomiques back/front possibles.
- Bibliothèques mutualisées (s2m-core, s2m-monetique) : dépôts séparés versionnés en SemVer.

## 3.2 Arborescence

```
s2m-<projet>/
|
|-- README.md
|-- CLAUDE.md                          # Conventions Claude Code (extrait
condensé)
|-- PROJECT_CONTEXT.md                 # État du projet (modules, décisions,
phase)
|-- package.json                       # Workspace pnpm
|-- pnpm-workspace.yaml
|-- tsconfig.base.json
|-- .env.example
|-- .nvmrc
|-- .editorconfig
|-- Makefile                           # make dev, test, build, migrate, lint,
clean
|-- docker-compose.yml                 # PG 18 + Redis 7 + NATS local
|
|-- backend/
|   |-- package.json
|   |-- tsconfig.json
|   |-- nest-cli.json
|   |-- Dockerfile
|   |-- CLAUDE.md                      # Règles spécifiques backend
|   +-- src/
|       |-- main.ts
|       |-- app.module.ts
|       |-- modules/                   # Un dossier par bounded context
|       |   +-- <module>/
|       |       |-- domain/            # Entités + règles métier pures
|       |       |-- application/       # Use-cases
|       |       |-- ports/             # Interfaces
|       |       |-- adapters/
|       |       |   |-- http/          # Controllers + mappers DTO
|       |       |   |-- postgres/      # Repositories Drizzle
|       |       |   |-- redis/
|       |       |   +-- nats/
|       |       +-- <module>.module.ts
|       |-- infrastructure/            # Plomberie transverse (pool DB, OTel)
|       +-- shared/                    # Helpers backend uniquement
```

```
|
|-- frontend/
|   |-- package.json
|   |-- tsconfig.json
|   |-- next.config.ts
|   |-- tailwind.config.ts
|   |-- Dockerfile
|   |-- CLAUDE.md
|   +-- src/
|       |-- app/                      # Next.js 16 App Router
|       |   |-- layout.tsx
|       |   |-- (auth)/
|       |   |-- (operator)/
|       |   |-- (admin)/
|       |   +-- (config)/
|       |-- components/
|       |   |-- ui/                   # shadcn/ui (modifiable avec validation)
|       |   |-- Brand/                # BrandLockup, SpxTile (depuis SELECT-PX)
|       |   +-- domain/               # Composants métier
|       |-- lib/
|       |   |-- api/                  # Couche accès données
|       |   |-- domain/               # Logique métier client
|       |   |-- brand.ts              # Config produit (env vars
NEXT_PUBLIC_PRODUCT_*)
|       |   +-- auth.ts
|       |-- hooks/
|       |   |-- queries/              # TanStack Query par domaine
|       |   +-- domain/               # Mapping DTO -> domain
|       |-- i18n/                     # Locales en/, fr/, ...
|       +-- styles/                   # globals.css avec @theme SELECT-PX
|
|-- shared/                           # Workspace partagé
|   |-- package.json
|   +-- src/
|       |-- schemas/                  # Schémas zod (source de vérité)
|       |-- types/                    # Types inférés
|       +-- constants/                # Codes erreur, devises
|
|-- migrations/                       # Flyway
|   |-- flyway.conf
|   |-- V1__initial_schema.sql
|   +-- V2__...
|
|-- deploy/
|   |-- docker/
|   |-- k8s/
|   |   |-- base/                     # Manifestes Kustomize
|   |   +-- overlays/                 # dev / staging / prod
|   +-- README.md
|
+-- docs/
    |-- architecture.md
    |-- getting-started.md
    |-- design/                       # Référence visuelle SELECT-PX
```

```
    |   |-- index.html                  # DS complet (tokens, components, install)
    |   +-- gallery.html                # 8 pages templates de référence
    +-- adr/                            # Architecture Decision Records
```

# 4. Règles pour Claude Code

## 4.1 Réflexe avant codage

- Lire CLAUDE.md racine puis CLAUDE.md du workspace concerné.
- Lire PROJECT_CONTEXT.md pour l'état actuel du projet.
- Scanner le module voisin pour patterns en place.
- Chercher dans s2m-core et s2m-monetique avant d'écrire (annexe A).
- Si aucun pattern existant ne couvre le besoin, demander avant d'introduire un nouveau pattern.

## 4.2 Règles absolues

**number natif sur montants**

Pour toute valeur monétaire ou financière, utiliser exclusivement decimal.js (via @s2m/core-decimal). Recherche `: number` dans domain/ doit retourner zéro résultat sur les modèles financiers.

**Cryptographie custom**

Aucune implémentation maison de HMAC, AES, RSA, signatures. Utiliser exclusivement @s2m/core-crypto.

**Audit sur mutations d'état**

Toute modification d'état critique (compte, transaction, configuration, utilisateur) appelle AuditRecorder. Inclut succès ET échecs critiques (champ outcome=SUCCESS|FAILURE).

**Erreurs typées**

Pas de `new Error("...")` ni `throw "string"`. Utiliser @s2m/core-errors. Code S2M-<DOMAIN>-<NUM>.

**Architecture hexagonale stricte**

- domain/ : aucune dépendance hors TypeScript et decimal.js
- application/ : dépend de domain et ports uniquement
- adapters/ : implémentent les ports, peuvent dépendre de tout sauf d'autres adapters
- Vérifié automatiquement par dependency-cruiser ou eslint-plugin-boundaries.

**console.log en code applicatif**

Utiliser le logger pino injecté via @s2m/core-observability. console.log toléré uniquement en débogage local temporaire.

**Sérialisation montants**

- Tout montant traverse le réseau en string, jamais en number.
- Schéma zod partagé valide deux champs liés : amount (string) + currency (ISO

4217).
- Décimales selon devise : JPY=0, EUR/USD=2, MAD/TND=3, KWD/BHD=3, BTC=8.
- Validator dédié : amountForCurrency (@s2m/core-validate).
- Conversion en Decimal à la frontière (controller backend ou couche API frontend) avant calcul.

**JWT : access en mémoire + refresh en cookie**

- Access token : 15 min, stocké en mémoire JS uniquement (jamais localStorage/sessionStorage).
- Access transmis via header Authorization: Bearer <token>.
- Refresh token : 7 jours, cookie HttpOnly Secure SameSite=Strict.
- Endpoint /auth/refresh pour renouvellement.
- Révocation serveur via Redis (jti blacklist).
- Identité utilisateur : endpoint /me, jamais parser le JWT côté client.

**Multi-tenant**

- Toute table métier : colonne tenant_id NOT NULL avec FK et index.
- Row Level Security PostgreSQL activée. Policy USING (tenant_id = current_setting('app.tenant_id')::uuid).
- Backend SET app.tenant_id en début de transaction depuis le contexte JWT.
- Garde applicatif : @s2m/core-tenant + decorator @CurrentTenant() injecté dans use-cases.
- Requête sans tenant -> erreur S2M-AUTH-002 avant DB.

## 4.3 Architecture frontend

- **app/, components/ui/** : présentation pure, sans logique métier ni appels réseau.
- **lib/domain/, hooks/domain/** : logique métier client (formatage, calculs d'affichage, mapping DTO->domain).
- **lib/api/, hooks/queries/** : exclusivement TanStack Query. Aucun fetch direct ailleurs.
- **components/domain/** : zone de jonction, consomme hooks queries + applique domain frontend.
- TanStack Query renvoie des DTO bruts. Mapping en objets de domaine (Decimal, Date) dans hooks/domain/. Composants consomment objets de domaine, jamais DTO bruts.

## 4.4 Design system SELECT-PX

- **Marque parente** : SELECT-PX, visible sur tous les portails S2M (LIC, PHS, MON, ACQ, CRD, ATM, etc.).
- **Branding par produit** : variables d'environnement NEXT_PUBLIC_PRODUCT_CODE (3-4 lettres uppercase), NEXT_PUBLIC_PRODUCT_NAME, NEXT_PUBLIC_PRODUCT_SUFFIX dans .env.local.

- **Tokens** : bloc @theme Tailwind 4 importé depuis @s2m/select-px-design dans app/globals.css. Couleurs brand (cyan-100/500, blue-600/900, ink), surfaces sombres (surface-0/1/2), sémantiques (success, warning, danger, info), typo (Montserrat display, Poppins sans, JetBrains Mono), radii (xs/sm/md/lg/xl), gradient signature.
- **Composants Brand** : BrandLockup, SpxTile importés depuis @s2m/select-px-design. Reproduisent tile sombre + chevron X + wordmark SELECT-PX / {CODE}_{SUFFIX}.
- **shadcn customisé** : composants pré-thémés via @s2m/select-px-design/spx-overrides. Installation : pnpm dlx shadcn@latest add button input badge table card dialog tabs sheet, puis appliquer overrides.
- **Référence visuelle** : docs/design/index.html (tokens, brand, install) et docs/design/gallery.html (8 pages templates : dashboard, liste, détail, etc.).
- **Polices** : importées via next/font/google (Montserrat 800, Poppins 300/500, JetBrains Mono).

**Pas d'invention visuelle**

- Aucune couleur, taille de police, radius, ombre hors des tokens du DS.
- Aucun composant UI custom hors de shadcn ou @s2m/select-px-design.
- Pour toute nouvelle page : partir d'un template de docs/design/gallery.html.
- Si un produit a besoin d'un accent distinct : prendre une couleur de la rampe brand existante, jamais une nouvelle teinte.
- En cas de doute visuel : lire docs/design/index.html sections 03 (tokens), 14 (stack), 15 (install), 16 (i18n).

## 4.5 Conventions de nommage

| Élément | Convention | Exemple |
|---|---|---|
| Classes | PascalCase | AccountRepository, CreateAccountUseCase |
| Ports (interfaces) | PascalCase via abstract class | export abstract class AccountRepository |
| Fonctions | camelCase | getAccountById, calculateFee |
| Constantes | UPPER_SNAKE_CASE | MAX_RETRY_ATTEMPTS |
| Fichiers TS | kebab-case | account.repository.pg.ts |
| Tables PostgreSQL | snake_case pluriel | accounts, ledger_entries |
| Colonnes DB | snake_case | created_at, tenant_id |
| Index | idx_<table>_<colonnes> | idx_accounts_tenant_id_status |
| Foreign keys | fk_<table>_<colonne> | fk_accounts_tenant_id |
| Routes REST | kebab-case pluriel + version | /v1/accounts, /v1/ledger-entries |

| Élément | Convention | Exemple |
|---|---|---|
| Query params | camelCase | ?accountStatus=ACTIVE&limit=50 |
| Codes erreur | S2M-<DOMAIN>-<NUM> | S2M-ACC-001, S2M-LED-042 |
| Schémas zod | PascalCase + Schema | AccountSchema |
| Types inférés | PascalCase | type Account = z.infer<typeof AccountSchema> |
| Variables env | UPPER_SNAKE_CASE | DATABASE_URL, REDIS_HOST |

## 4.6 Qualité du code

- Une fonction = une responsabilité. >30 lignes = signal de scission.
- Noms explicites long > noms courts cryptiques.
- Commentaires : pourquoi, jamais quoi. Uniquement pour choix non triviaux.
- Early returns plutôt que grandes imbrications if/else.
- const par défaut, immutabilité préférée.
- Pas d'héritage profond, pas de proxies, pas d'injection magique.
- Tests obligatoires : unitaires sur domain/, intégration sur application/. Couverture ≥80%.

## 4.7 Discipline IA

- str_replace pour modifications ciblées, pas de réécriture totale.
- view_range pour lecture partielle des longs fichiers.
- Pas de ré-énoncé du contenu fourni : référencer par chemin et ligne.
- Réponses concises, pas de préambule ni conclusion verbeuse.

**Invention et duplication**

- Utiliser strictement la nomenclature du référentiel et du code existant.
- Pas de synonymes inventés (compte ≠ wallet ≠ account record).
- Pas de nouveau composant, module ou concept sans validation explicite.
- Pas de duplication d'un libellé sous deux noms différents.
- Pas de schéma décoratif. Chaque visuel = réalité technique vérifiable dans le code.
- En cas de doute : [à valider] entre crochets, jamais d'approximation.

## 4.8 Mise à jour contexte

- Toute évolution structurelle (nouveau module, convention, dépendance, refactor) met à jour CLAUDE.md du workspace concerné.
- PROJECT_CONTEXT.md du projet est mis à jour à chaque session significative.
- **Feedback loop** : quand Claude Code se trompe, consigner la correction dans CLAUDE.md immédiatement.

- Changement généralisable à d'autres projets S2M : remonter vers s2m-core ou s2m-monetique.

## 4.9 Workflow d'une tâche

```
1. Lire CLAUDE.md racine + workspace + PROJECT_CONTEXT.md
2. Identifier le module concerné (modules/<X>/)
3. Scanner code voisin pour patterns
4. Chercher dans @s2m/core et @s2m/monetique (annexe A)
5. Implémenter en respectant l'hexagonale
6. Ajouter schémas zod dans shared/ si contrat API
7. Écrire tests Vitest (domain) + Supertest (HTTP)
8. Si schéma DB : créer migration Vxxx__*.sql
9. make lint (typecheck + boundaries + gitleaks)
10. Mettre à jour CLAUDE.md / PROJECT_CONTEXT.md si nécessaire
```

## 4.10 Commandes unifiées

| Commande | Effet |
|---|---|
| make dev | Démarre backend + frontend + datastores Docker |
| make test | Tests unitaires + intégration + E2E |
| make build | Build production de tous les workspaces |
| make lint | Lint + typecheck + boundaries + gitleaks |
| make migrate | Applique migrations Flyway |
| make clean | Nettoie builds et arrête conteneurs |

## 4.11 Matrice patterns autorisés

| Élément | Couche autorisée | Couches interdites |
|---|---|---|
| Règle métier pure | domain/ | application/, adapters/, controllers |
| Orchestration use-case | application/ | domain/, adapters/ |
| Transaction DB | application/ via repository | domain/, controllers |
| Interface repository | ports/ | adapters/, controllers |
| Implémentation repository | adapters/postgres/ | domain/, application/ |
| Validation payload entrant | adapters/http/ via zod | domain/, application/ |
| Publication NATS | adapters/nats/ | controllers, domain/, application/ |

| Élément | Couche autorisée | Couches interdites |
|---|---|---|
| | | direct |
| Lecture/écriture Redis | adapters/redis/ | domain/, application/ direct |
| Mapping DTO ↔ Domain | adapters/http/mappers/ | domain/, controllers |
| Gestion d'erreur | @s2m/core-errors | new Error(), throw string |

**Dossiers interdits dans modules backend**

services/, managers/, helpers/, utils/, common/, lib/ INTERDITS dans backend/src/modules/<X>/. Toute logique va dans domain/, application/, ports/, adapters/. Note : lib/ et hooks/ autorisés à la racine du frontend (convention Next.js).

## 4.12 Templates obligatoires

### 4.11.1 Use-case

```
// modules/account/application/create-account.usecase.ts
import { Injectable } from '@nestjs/common';
import { Account } from '../domain/account.entity';
import { CreateAccountInput } from '../domain/account.types';
import { AccountRepository } from '../ports/account.repository';
import { AuditRecorder } from '../ports/audit.recorder';

@Injectable()
export class CreateAccountUseCase {
  constructor(
    private readonly accountRepository: AccountRepository,
    private readonly auditRecorder: AuditRecorder,
  ) {}

  async execute(input: CreateAccountInput): Promise<Account> {
    const account = Account.create(input);
    await this.accountRepository.save(account);
    await this.auditRecorder.record({
      action: 'ACCOUNT_CREATED',
      resourceType: 'account',
      resourceId: account.id,
      outcome: 'SUCCESS',
      payload: { tenantId: account.tenantId, productId: account.productId },
    });
    return account;
  }
}
```

### 4.11.2 Port et adapter

```
// modules/account/ports/account.repository.ts
import { Account } from '../domain/account.entity';

export abstract class AccountRepository {
  abstract save(account: Account): Promise<void>;
  abstract findById(id: string): Promise<Account | null>;
  abstract findByTenant(tenantId: string, page: PageOpts): Promise<Account[]>;
}

// modules/account/adapters/postgres/account.repository.pg.ts
import { Injectable } from '@nestjs/common';
import { AccountRepository } from '../../ports/account.repository';
import { Account } from '../../domain/account.entity';
import { toDomain, toRow } from './account.mapper';

@Injectable()
export class AccountRepositoryPg extends AccountRepository {
  constructor(private readonly db: Database) { super(); }

  async save(account: Account): Promise<void> {
```

```
    await this.db.insert(accountsTable).values(toRow(account));
  }

  async findById(id: string): Promise<Account | null> {
    const row = await this.db.select({ id: accountsTable.id, /* champs explicites
*/ })
      .from(accountsTable).where(eq(accountsTable.id, id)).limit(1);
    return row[0] ? toDomain(row[0]) : null;
  }
}
```

### 4.11.3 Mapping DTO ↔ Domain

```
// adapters/http/mappers/account.mapper.ts
import { Account } from '../../../domain/account.entity';
import { AccountDTO } from '@s2m-uams/shared';
import { Decimal } from 'decimal.js';

export function toDomain(dto: AccountDTO): Account {
  return Account.rehydrate({
    id: dto.id,
    tenantId: dto.tenantId,
    balance: new Decimal(dto.balance),    // string -> Decimal
    currency: dto.currency,
    status: dto.status,
    createdAt: new Date(dto.createdAt),
  });
}

export function toDTO(account: Account): AccountDTO {
  return {
    id: account.id,
    tenantId: account.tenantId,
    balance: account.balance.toFixed(account.currency.decimals),  // selon devise
    currency: account.currency.code,
    status: account.status,
    createdAt: account.createdAt.toISOString(),
  };
}
```

### 4.11.4 Controller HTTP

```
// modules/account/adapters/http/account.controller.ts
import { Controller, Post, Body, UseGuards } from '@nestjs/common';
import { AuthGuard, CurrentTenant } from '@s2m/core-auth';
import { CreateAccountUseCase } from '../../application/create-account.usecase';
import { CreateAccountSchema, AccountDTO } from '@s2m-uams/shared';
import { ZodValidationPipe } from '@s2m/core-validate';
import { toDTO } from './mappers/account.mapper';

@Controller('v1/accounts')
@UseGuards(AuthGuard)
export class AccountController {
  constructor(private readonly createAccount: CreateAccountUseCase) {}
```

```
  @Post()
  async create(
    @Body(new ZodValidationPipe(CreateAccountSchema)) dto: CreateAccountInputDTO,
    @CurrentTenant() tenantId: string,
  ): Promise<AccountDTO> {
    const account = await this.createAccount.execute({ ...dto, tenantId });
    return toDTO(account);
  }
}
```

### 4.11.5 Tests

```
// modules/account/domain/__tests__/account.spec.ts
import { describe, it, expect } from 'vitest';
import { Account } from '../account.entity';

describe('Account', () => {
  it('crée un compte ACTIVE avec balance zero', () => {
    const account = Account.create({
      tenantId: 't-001', productId: 'p-001', currency: 'MAD',
    });
    expect(account.status).toBe('ACTIVE');
    expect(account.balance.equals(0)).toBe(true);
  });
});

// modules/account/__tests__/account.e2e.spec.ts
import request from 'supertest';

describe('POST /v1/accounts', () => {
  it('crée un compte (201)', async () => {
    const response = await request(app.getHttpServer())
      .post('/v1/accounts')
      .set('Authorization', `Bearer ${token}`)
      .send({ productId: 'p-001', currency: 'MAD' })
      .expect(201);
    expect(response.body.balance).toBe('0.000');
  });
});
```

## 4.13 Flow end-to-end

```
Frontend (React + TanStack Query)
  | POST /v1/accounts
  | Body : DTO en JSON, montants en string
  | Header : Authorization: Bearer <token> + traceparent
  v
Controller (adapters/http)
  | 1. Guard JWT -> identité, tenant, rôles
  | 2. ZodValidationPipe -> validation runtime
  | 3. @CurrentTenant() injecte tenant_id
  | 4. Appel use-case avec input typé
```

```
  v
Use-case (application)
  | 5. Account.create() -> entité du domaine
  | 6. Transaction PostgreSQL ouverte (SET app.tenant_id)
  | 7. Repository.save() via port -> adapter Drizzle
  | 8. AuditRecorder.record() dans la même transaction
  | 9. Outbox event écrit (atomique avec écriture métier)
  | 10. Commit
  v
Outbox relay (worker async)
  | 11. Polling LIMIT 100 FOR UPDATE SKIP LOCKED
  | 12. Publie sur NATS JetStream
  | 13. Marque published_at
  v
Consommateurs NATS (webhooks, EOD, autres modules)
  | 14. Reçoivent avec acknowledgement explicite
  v
Controller renvoie DTO via toDTO()
  | Status 201, montants en string, dates ISO 8601
  v
Frontend (TanStack Query)
  | queryClient.invalidateQueries
```

## 4.14 Performance

**Pagination cursor-based**

Toutes les listes paginées : ?cursor=<uuidv7>&limit=50 (max 200). Pas d'offset/limit.

**SELECT ***

Colonnes explicitement listées. Drizzle impose ce style par construction (.select({ id: t.id, ... })).

**Index obligatoires**

Toute foreign key porte un index. Toute colonne dans WHERE/ORDER BY récurrent porte un index. Index créés dans le même fichier de migration que la table.

**Timeouts explicites**

- Redis : 1 seconde
- NATS publish : 5 secondes
- HTTP externe : 10 secondes + circuit breaker
- PostgreSQL statement : 30 secondes
- Job EOD long : 30 minutes max, sinon fragmenter

## 4.15 Sécurité

**Rate limiting**

- Endpoints publics : 100 req/min/IP, 1000 req/min/user.
- Endpoints sensibles (login, password reset) : 5/min + backoff exponentiel.
- Implémenté via @nestjs/throttler avec store Redis partagé.

**Headers et CORS**

- Content-Type vérifié pour POST/PUT/PATCH (application/json strict).
- CORS : allowlist explicite, jamais *.
- CSRF token obligatoire pour opérations sensibles depuis cookie.

**Sanitization**

- zod fait validation type/format/longueur.
- DOMPurify côté frontend pour toute chaîne affichée.
- SQL : paramètres positionnels uniquement (Drizzle garantit par construction).

**Détection secrets**

- gitleaks en pre-commit hook (Husky) + CI.
- Trufflehog scanne périodiquement l'historique.
- Tout secret committé -> rotation immédiate des credentials.

## 4.16 Versioning, dates, i18n

**API versioning**

Préfixe URL /v1/, /v2/. Jamais via header. Support N-1 pendant 18 mois après release N+1. Breaking changes documentés dans CHANGELOG-API.md.

**Dates UTC**

- Stockage : PostgreSQL TIMESTAMPTZ obligatoire.
- Sérialisation réseau : ISO 8601 (2026-04-30T15:32:00.000Z).
- Manipulation backend : Temporal (polyfill @js-temporal/polyfill) ou Date natif.
- Conversion fuseau local : frontend uniquement, jamais backend.

**Internationalisation**

- Locales par défaut : EN + FR (en, fr).
- Architecture extensible : toute autre locale ajoutable via fichier de traduction (ar, es, de, ar-MA RTL, etc.) sans modifier le code.
- Bibliothèque frontend : next-intl.
- Codes erreur S2M-<DOMAIN>-<NUM> traduits dans @s2m-<projet>/shared.
- Libellés métier dans @s2m/monetique-i18n.
- RTL géré nativement quand locale RTL activée.
- Fichiers EOD restent en formats normalisés (ISO 20022, AFB) indépendants de la

locale.

## 4.17 Outillage repo

**Standards à la racine**

- .nvmrc avec version Node exacte.
- .editorconfig pour cohérence multi-IDE.
- Husky + lint-staged en pre-commit (lint, typecheck, gitleaks).
- Biome ou ESLint+Prettier (configuration @s2m/eslint-config).
- Renovate ou Dependabot pour mises à jour de dépendances.
- Conventional commits + commitlint.
- CHANGELOG.md généré automatiquement (release-please).

## 4.18 Outbox

**Implémentation polling applicatif**

- Table outbox : (id, aggregate_id, event_type, payload, created_at, published_at NULL).
- Worker NestJS dédié (séparé des controllers).
- Polling toutes les 200 ms : LIMIT 100, FOR UPDATE SKIP LOCKED.
- Publication NATS, puis published_at = NOW().
- Métrique : âge max événements non publiés. Alerte > 5 secondes.
- Pas de Debezium/CDC tant que volume < 10k events/seconde.

## 4.19 CLAUDE.md projet

- Reste sous 300 lignes. Format extrait du référentiel.
- Contenu :
- - Identité projet et stack (5 lignes)
- - Lien vers ce référentiel
- - Règles ABSOLUES (callouts MUST/MUST NOT critiques)
- - Conventions spécifiques au projet (modules, codes erreur)
- - Workflow standard
- - Liens vers templates dans docs/templates/

# 5. Checklist nouveau projet

## 5.1 Setup

- Cloner s2m-service-template.
- Renommer projet (package.json, README).
- Configurer .env, .nvmrc.
- make dev pour vérifier.
- Créer dépôt Git, protections de branche.
- Configurer CI (GitHub Actions ou GitLab CI).

## 5.2 Domaine métier

- Identifier bounded contexts -> modules/<X>/.
- Définir schémas zod dans shared/src/schemas/.
- Créer migrations Flyway initiales.
- Adapter CLAUDE.md racine au domaine.
- Initialiser PROJECT_CONTEXT.md.

## 5.3 Validation

- Versions dépendances alignées avec ce référentiel.
- Aucun float natif sur modèles financiers.
- Routes REST en kebab-case pluriel + version /v1/.
- Codes erreur format S2M-<DOMAIN>-<NUM>.
- Toute mutation appelle AuditRecorder.
- Multi-tenant : tenant_id + RLS + @CurrentTenant() en place.
- Couverture tests ≥80% sur domain/ et application/.
- Enforcement hexagonal actif (dependency-cruiser ou eslint-plugin-boundaries).

# Annexe A. Catalogue @s2m/core et @s2m/monetique

- Briques mutualisées entre tous les projets S2M.
- Claude Code DOIT les utiliser au lieu de réimplémenter localement.
- Besoin non couvert : proposition d'évolution avant implémentation locale.

## A.1 @s2m/core

| Package | Rôle |
|---|---|
| @s2m/core-crypto | HMAC-SHA256, signatures. Aucune crypto custom hors de ce package. |
| @s2m/core-audit | Recorder d'audit immuable, scellement HMAC chaîné, append-only. |
| @s2m/core-decimal | Wrapper decimal.js avec helpers monétaires (formatage, arrondis, ISO 4217). |
| @s2m/core-errors | Erreurs typées (codes S2M-<DOMAIN>-<NUM>, messages i18n). |
| @s2m/core-validate | Validators ISO : IBAN, BIC, MCC, currency 4217, country 3166, amountForCurrency. |
| @s2m/core-auth | Guards NestJS, JWT (access mémoire + refresh cookie), revocation Redis. |
| @s2m/core-tenant | Multi-tenancy : extraction tenant JWT, propagation contexte, decorator @CurrentTenant(). |
| @s2m/core-idempotency | Idempotency via Redis SetNX, TTL configurable, fingerprint requête. |
| @s2m/core-observability | OTel SDK + pino préconfigurés + métriques standard. |
| @s2m/core-comm-http | Client HTTP avec retry, circuit-breaker, timeout, OTel. |
| @s2m/core-comm-nats | Wrapper nats.js : publish/subscribe, durable consumers, JetStream. |
| @s2m/core-comm-webhook | Dispatcher webhook signé, retry exponentiel, dead-letter. |
| @s2m/core-redis | Wrapper ioredis : circuit-breaker, lock manager (Redlock), pub/sub. |
| @s2m/core-testkit | DB éphémères, fixtures, snapshots, mocks NATS/Redis pour Vitest. |
| @s2m/eslint-config | Configuration ESLint partagée + eslint-plugin-boundaries. |
| @s2m/select-px-design | Design system parent S2M. Tokens @theme Tailwind 4, |

| Package | Rôle |
|---|---|
| | composants Brand (BrandLockup, SpxTile), shadcn customisé (spx-overrides), 8 pages templates. Voir §4.4. |

## A.2 @s2m/monetique

| Package | Rôle |
|---|---|
| @s2m/monetique-ledger | Écritures partie double, scellement HMAC chaîné, helpers replay/snapshot. |
| @s2m/monetique-tx-codes | Catalogue codes transaction S2M, accounting rules paramétrables. |
| @s2m/monetique-eod-reader | Parseurs PAYROLL CSV/TSV/largeur fixe, détection format, validation. |
| @s2m/monetique-eod-writer | Générateurs CLEARING ISO 20022, AFB, GIM-UEMOA, HPS. |
| @s2m/monetique-eod-signer | Signature HMAC ou GPG sur fichiers EOD, vérification. |
| @s2m/monetique-velocity | Compteurs vélocité multi-fenêtre via Redis. |
| @s2m/monetique-reconciliation | Engine de réconciliation Float, détection écarts, alerting. |
| @s2m/monetique-i18n | Dictionnaires i18n libellés métier (statuts, types, codes erreur). EN + FR + extensible. |

**Cycle de releases**

- SemVer strict.
- PATCH (fix, sécurité) : propagé automatiquement via Renovate.
- MINOR (feature rétrocompatible) : mise à jour planifiée trimestrielle.
- MAJOR (breaking change) : migration manuelle, changelog détaillé, fenêtre de support N-1 18 mois.